

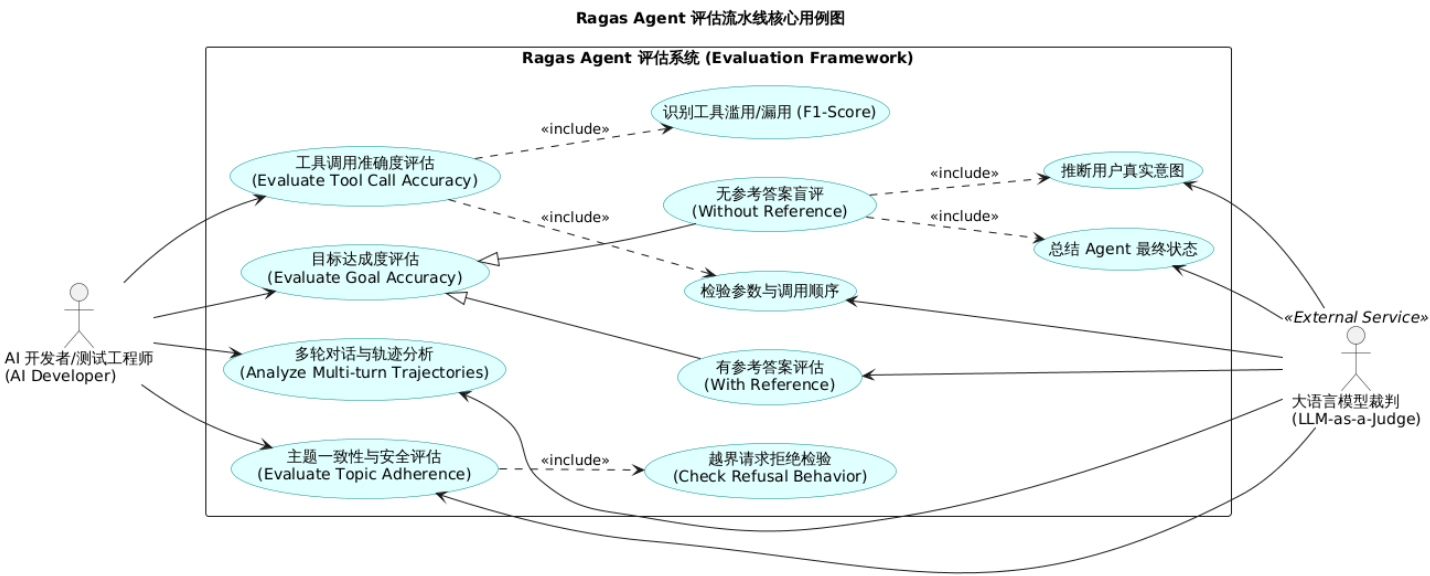
Ragas项目分析文档

分工情况

- 周亦余：Agent 目标准确度的详细设计、其他核心 Agent 评估指标的分析、提升 Python 高质量开源项目的阅读与分析能力
- 王传鹏：Ragas 及 Agent 评估功能描述、Ragas 的软件架构及各个包和类的作用、软件功能与类间的对应关系、对开源协作与代码规范的认识
- 刘佳昱：标注范围与工作量统计、代码标注示例与规范、使用的大模型辅助工具、大模型辅助开源阅读的成效、遇到的问题与反思
- 刘怡然：Agent 评估的整体执行流水线，工具调用准确度的详细设计、深入理解大模型 Agent 的评测理论与工程化实现

1. Ragas 开源软件的代码泛读

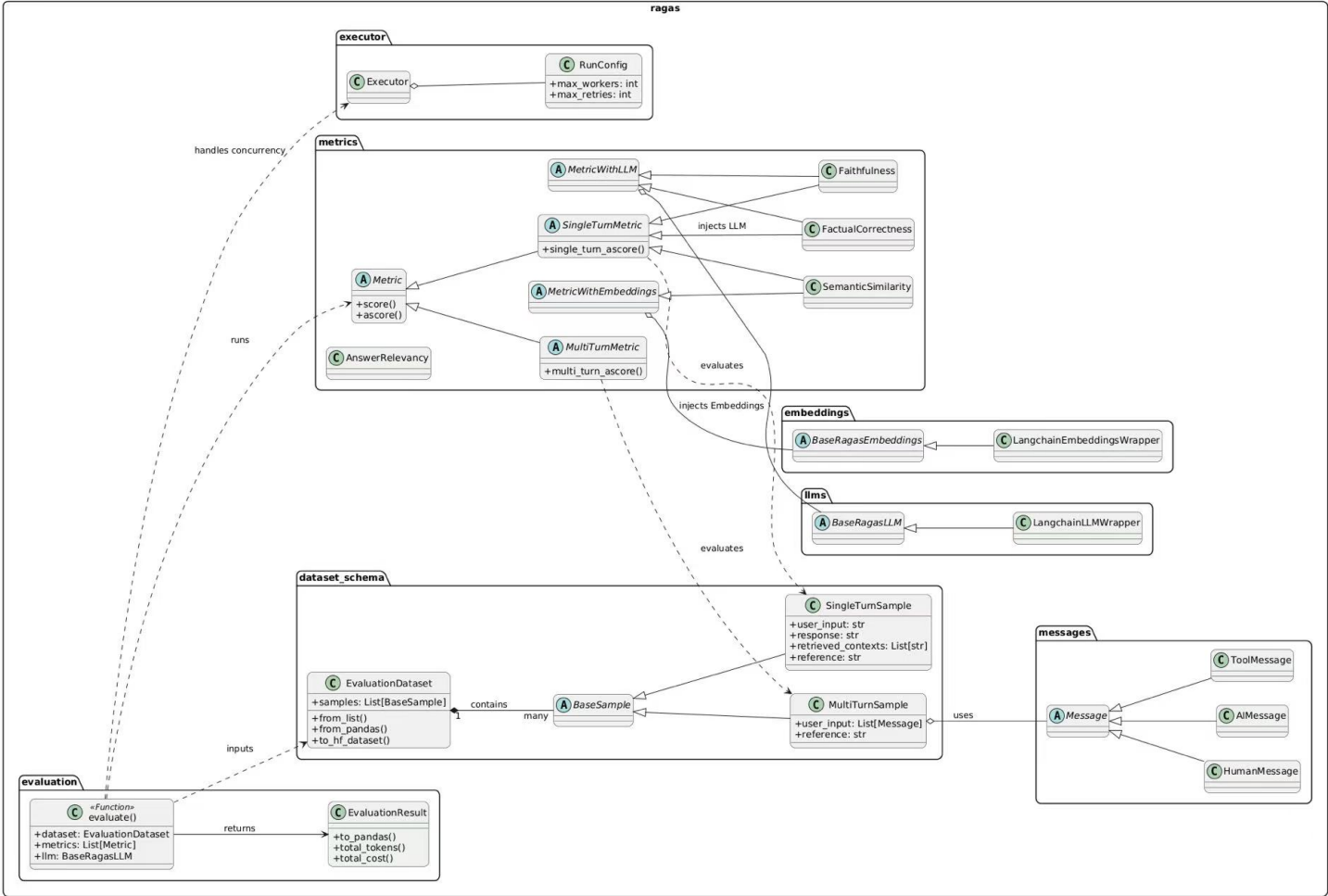
1.1 Ragas 及 Agent 评估功能描述



在开发实际 Agent 系统的过程中，随着执行链条的变长，大模型固有的不确定性会导致“幻觉”、“死循环”和“工具滥用”等问题被逐级放大。而 Ragas (Retrieval Augmented Generation Assessment) 提供的正是一套自动化、数据驱动的评估流水线，用于诊断和衡量这些问题。Ragas 最初是一个致力于评估 RAG（检索增强生成）系统性能的开源框架。随着大模型技术的演进，其能力已显著扩展，目前已全面支持复杂的大模型 Agent（智能体）系统的多维度量化评估。在 Agent 评估领域，Ragas 所注重的核心评估项主要包括：

- 1. **多轮对话与轨迹分析。**不同于传统的单轮问答，Ragas 能够处理 Agent 在解决复杂任务过程中产生的多轮交互（Multi-turn）轨迹，能够全面解析 Agent 的自主感知、规划与执行全过程。
- 2. **工具调用准确度评估。**精确衡量 Agent 在多轮推理中是否根据用户意图正确调用了外部工具，并检验参数传递的准确性与调用顺序。同时通过 F1-Score 综合评估工具调用的全面性与准确度，有效识别“工具滥用”或“漏用”。
- 3. **目标达成度盲评与比对。**提供有参考答案（With Reference）和无参考答案（Without Reference）两种模式。通过将 LLM 作为裁判，推断用户的真实意图并总结 Agent 的最终执行状态，从而判定 Agent 是否真正解决或达成了用户的根本需求。
- 4. **主题一致性与业务安全。**评估 Agent 在对话交互过程中是否严格遵守预定义的主题边界，以及面对越界或不相关的请求时是否能进行适当的拒绝，确保 Agent 在生产环境中的安全防线。

1.2 Ragas 的软件架构及各个包和类的作用



Ragas 的整体架构贯彻了模块化和分层的软件设计，并保持了高度解耦。这不仅提升了框架在大规模评测时的稳定性，还通过践行“开闭原则”保障了高度的可扩展性。其主要层级及其包的作用如下：

- 1. **核心调度与执行层 (ragas.evaluation, ragas.executor, ragas.experiment, ragas.run_config)**
这一层负责整体评估任务的触发与高并发控制。ragas.evaluation 提供统一 API 入口 aevaluate()，返回 EvaluationResult。ragas.executor 利用 asyncio 构建并发任务流水线，解决

I/O 阻塞问题。ragas.experiment 与 ragas.run_config 负责评测实验环境的高级配置管理。

2. 现代指标抽象与业务实现层 (ragas.metrics)

ragas.metrics.collections 以现代组件架构（Modern Component Architecture）提供各类业务评估指标（如 ToolCallAccuracy、AgentGoalAccuracyWithReference 等）的独立解耦实现，底层基于 ragas.metrics.base.BaseMetric 统一契约。

3. 大模型基础设施与第三方集成层 (ragas.llms, ragas.embeddings, ragas.integrations)

ragas.llms 与 ragas.embeddings 负责对不同底层大模型与向量服务（如 OpenAI, HuggingFace等）进行标准化封装（BaseRagasLLM、BaseRagasEmbeddings），提供异步生成与嵌入接口。ragas.integrations 负责与 LangChain, LlamaIndex 等外部框架及监控体系（如 Langsmith, Opik）的对接。

4. 数据协议与持久化层 (ragas.dataset, ragas.dataset_schema, ragas.messages, ragas.backends)

采用 Pydantic 构建强类型约束数据体。ragas.messages 定义细粒度对话抽象（HumanMessage、ToolCall等），ragas.dataset_schema 将其组装为 MultiTurnSample 和 EvaluationDataset。结构化测试数据可通过 ragas.backends 持久化至本地 CSV、JSONL 或云端。

5. 提示词与自动化优化层 (ragas.prompt, ragas.optimizers)

ragas.prompt 定义了面向 LLM 的复杂交互提示词模板（如 PydanticPrompt 和 PromptMixin），而 ragas.optimizers 提供了自动化提升 Prompt 效果的策略实现（如 DSPy Optimizer 适配器和遗传算法优化）。

6. 合成测试集生成层 (ragas.testset)

Ragas 不仅拥有评测能力，还在这一层内置了数据生成引擎。通过 ragas.testset.graph 等模块从语料中自动化生成多样化的问答测试集（含知识图谱解析、分布变换与合成机制）。

7. 可观测性与辅助设施层 (ragas.callbacks, ragas.cost, ragas.cache, ragas.cli, ragas.utils 等)

为评测流水线提供运行支撑：ragas.cost 负责算力 Token 计费统计，ragas.callbacks 暴露执行钩子以对接监控，ragas.cache 进行调用缓存防重，ragas.cli 提供终端命令行支持，其余诸如 exceptions、validation 规范错误与入参。

1.3 软件功能与类间的对应关系

根据对 Ragas 开源软件源码的阅读、理解与标注，其 Agent 评估的核心软件功能与代码类/方法之间的对应关系如下表 1 所示：

序号	功能名称	实现包/模块	核心实现类与方法
1	评估任务总入口与流水线触发	ragas.evaluation	aevaluate() 顶层函数，返回 EvaluationResult 对象

2	异步并发控制与结果回收	ragas.executor	Executor 调度类及其 submit(), areresults() 方法
3	交互消息实体的基元定义	ragas.messages	HumanMessage, AIMessage, ToolMessage, ToolCall 等离散实体
4	多轮对话数据样本的标准化组装	ragas.dataset_schema	MultiTurnSample 类与 EvaluationDataset 评测集基类
5	大模型裁判的统一调用抽象	ragas.llms.base	BaseRagasLLM 抽象类及其 agenerate_text(), agenerate() 方法
6	向量模型的提取与转换抽象	ragas.embeddings.base	BaseRagasEmbeddings 抽象类及其 embed_text(), embed_documents()
7	基于 Pydantic 的提示词生成器	ragas.prompt	PydanticPrompt 类及其 format() 与自动纠错流
8	提示词 DSPy 及遗传算法全自动优化	ragas.optimizers	DSPyOptimizer 及等算法策略类
9	合成评估测试集的图结构引擎	ragas.testset.graph	KnowledgeGraph 与相关 transforms 演化图管道
10	外部生态监控与分析平台对接	ragas.integrations	向 langchain, langsmith, opik 导出的专属回调挂载类
11	模型评测开销监控与 Token 统计	ragas.cost	CostCallbackHandler 类及其 total_tokens(), total_cost() 统计方法
12	评估工具调用的精确度计算	ragas.metrics.collections.tool_call_accuracy	ToolCallAccuracy 类及其 ascore() 评估核心控制流
13	基于参考的 Agent 目标达成度评估	ragas.metrics.collections.agent_goal_accuracy	AgentGoalAccuracyWithReference 类及状态推理逻辑 _infer_goal_outcome()
14	无参考下 Agent 目标盲评推断	ragas.metrics.collections.agent_goal_accuracy	AgentGoalAccuracyWithoutReference 类
15	检测多轮对话的主题一致性与越界拒绝	ragas.metrics.collections.topic_adherence	TopicAdherence 类及主题推断流 _extract_topics()

2. Ragas 开源软件的代码标注情况

2.1 标注范围与工作量统计

在本研究中，我们针对ragas源码和评估示例中的中共 33 个包和 54 个文件进行了详细的代码标注。这些标注涵盖了基于dataset和experiment的评估流程和基于metric的评估流程。这些代码注释的详细信息如下表所示。

通过这些注释，我们可以更好地理解每个文件（类、函数）和包的设计意图、功能实现以及与其他部分的交互逻辑。这些信息不仅有助于我们更好地理解和运用现有代码，还可以为后续的功能扩展提供有力的支持。

序号	模块/包路径	文件名称	类/函数名称	标注的代码行数
1	examples/ragas_examples/rag_eval	rag.py	default_rag_client	10
2		evals.py	load_dataset, run_experiment, main	19
3	examples/ragas_examples/agent_evals	agent.py	MathToolsAgent, _execute_tool_call, solve, get_default_agent	26
4		evals.py	correctness_metric, load_dataset, run_experiment, main	26
5	examples/ragas_examples/llamaindex_agent_evals	llamaindex_agent.py	add_item, remove_item, list_items, workflow	11
6		evals.py	tool_call_accuracy_metric, goal_accuracy_metric_without_llm, load_dataset, run_experiment, main	24
7	examples/ragas_examples/text2sql	text2sql_agent.py	Text2SQLAgent	11
8		evals.py	execution_accuracy, text2sql_experiment, load_dataset	15
9	examples/ragas_examples/ag_ui_agent_experiments	experiments.py	load_scientist_dataset, create_evaluator_components, scientist_experiment, run_tool_experiment, main	23

10	src/ragas/backends	init.py	模块导出 (GDriveBackend 可选)	11
11		base.py	BaseBackend	15
12		gdrive_backend.py	GDriveBackend, _setup_auth, _ensure_folder_structure, _load, _save	14
13		inmemory.py	InMemoryBackend	9
14		local_csv.py	LocalCSVBackend	10
15		local_jsonl.py	LocalJSONLBackend, _serialize_datetime, _deserialize_datetime	11
16		registry.py	BackendRegistry, get_registry	15
17		utils.py	工具函数	9
18		dataset.py	DataTable, Dataset	13
19	src/ragas	dataset_schema.py	BaseSample	5
20		experiment.py	ExperimentProtocol, ExperimentWrapper, experiment	29
21				
21	src/ragas/metrics	init.py	模块导出	9
22		base.py	MetricOutputType, Metric, MetricWithLLM, SingleTurnMetric, SimpleBaseMetric, SimpleLLMMetric	36
23		decorator.py	create_metric_decorator, CustomMetric	9
24		discrete.py	DiscreteMetric, discrete_metric	7
25		numeric.py	NumericMetric, numeric_metric	6
26		ranking.py		6

			RankingMetric, ranking_metric	
27		result.py	MetricResult	10
28		validators.py	DiscreteValidator, NumericValidator, RankingValidator, get_validator_for_allowed_val ues	10
29	src/ragas/metrics/collections	init.py	模块导出	6
30		base.py	BaseMetric	9
31	src/ragas/metrics/collections/agent _goal_accuracy	metric.py	AgentGoalAccuracyWithRefere nce	4
32	src/ragas/metrics/collections/answ er_accuracy	metric.py	AnswerAccuracy	5
33	src/ragas/metrics/collections/answ er_correctness	metric.py	AnswerCorrectness	7
34	src/ragas/metrics/collections/answ er_relevancy	metric.py	AnswerRelevancy	9
35	src/ragas/metrics/collections/chrf_s core	metric.py	CHRFscore	3
36	src/ragas/metrics/collections/conte xt_entity_recall	metric.py	ContextEntityRecall	5
37	src/ragas/metrics/collections/conte xt_precision	metric.py	ContextPrecisionWithReferenc e, ContextPrecisionWithoutRefer ence	8
38	src/ragas/metrics/collections/conte xt_recall	metric.py	ContextRecall	7
39	src/ragas/metrics/collections/conte xt_relevance	metric.py	ContextRelevance	6
40	src/ragas/metrics/collections/datac ompy_score	metric.py	DataCompyScore	6
41	src/ragas/metrics/collections/doma in_specific_rubrics	metric.py	DomainSpecificRubrics	4

42	src/ragas/metrics/collections/factual_correctness	metric.py	FactualCorrectness	7
43	src/ragas/metrics/collections/faithfulness	metric.py	Faithfulness	13
44	src/ragas/metrics/collections/instance_specific_rubrics	metric.py	InstanceSpecificRubrics	4
45	src/ragas/metrics/collections/multi_modal_faithfulness	metric.py	MultiModalFaithfulness	6
46	src/ragas/metrics/collections/multi_modal_relevance	metric.py	MultiModalRelevance	6
47	src/ragas/metrics/collections/noise_sensitivity	metric.py	NoiseSensitivity	7
48	src/ragas/metrics/collections/quoted_spans	metric.py	QuotedSpansAlignment	4
49	src/ragas/metrics/collections/response_groundedness	metric.py	ResponseGroundedness	6
50	src/ragas/metrics/collections/sql_semantic_equivalence	metric.py	SQLSemanticEquivalence	4
51	src/ragas/metrics/collections/summary_score	metric.py	SummaryScore	8
52	src/ragas/metrics/collections/tool_call_accuracy	metric.py	ToolCallAccuracy	9
53	src/ragas/metrics/collections/tool_call_f1	metric.py	ToolCallF1	7
54	src/ragas/metrics/collections/topic_adherence	metric.py	TopicAdherence	7

2.2 代码标注示例与规范

我们截取了ragas进行并发处理数据集的关键逻辑，下面是对其进行代码标注的展示。

experiment.py

```

1  """Experiments hold the results of an experiment against a dataset."""
2  """

```



```

3  __all__ 指明了当使用 from module import * 语句时，哪些名称会被导入到当前命名空间中。在
   这个例子中，__all__ 包含了 "DataTable" 和 "Dataset"，
4  这意味着执行 from ragas.dataset import * 时，只有 DataTable 和 Dataset 会被导入，
5  而其他定义在该模块中的名称将不会被导入。
6  这是一种控制模块接口的方式，确保用户只能访问你希望他们访问的部分。同时避免名称冲突
7  """
8
9  __all__ = ["Experiment", "experiment", "version_experiment"]
10
11
12  """
13  arun返回值写成"Experiment"的用意是什么？
14  为什么不直接写成Experiment呢？
15  """
16
17  @t.runtime_checkable
18  class ExperimentProtocol(t.Protocol):
19      async def __call__(self, *args, **kwargs) -> t.Any: ...
20      async def arun(
21          self,
22          dataset: Dataset,
23          name: t.Optional[str] = None,
24          backend: t.Optional[t.Union[BaseBackend, str]] = None,
25          *args,
26          **kwargs,
27      ) -> "Experiment": ...
28
29  """
30  t.Type 的作用是把类型语义从“对象”改成“类”
31  t.Type[BaseModel] 表示“BaseModel类或其子类的类型”，而不是一个BaseModel实例。
32  这在这里很重要，因为我们需要传递一个模型类（例如一个Pydantic模型类）来定义实验结果的结构，而不是一个模型实例。
33  """
34
35  class ExperimentWrapper:
36      """Wrapper class that implements ExperimentProtocol for decorated
   functions."""
37
38      def __init__(
39          self,
40          func: t.Callable,
41          experiment_model: t.Optional[t.Type[BaseModel]] = None,
42          default_backend: t.Optional[t.Union[BaseBackend, str]] = None,
43          name_prefix: str = "",
44      ):
45          self.func = func
46          self.experiment_model = experiment_model

```

```

47     self.default_backend = default_backend
48     self.name_prefix = name_prefix
49     # Preserve function metadata
50     self.__name__ = getattr(func, "__name__", "experiment_function")
51     self.__doc__ = getattr(func, "__doc__", None)
52
53     async def __call__(self, *args, **kwargs) -> t.Any:
54         """Call the original function."""
55         if asyncio.iscoroutinefunction(self.func):
56             return await self.func(*args, **kwargs)
57         else:
58             return self.func(*args, **kwargs)
59
60     """
61     *args 和 **kwargs 是 Python 中的特殊语法，用于在函数定义中表示可变数量的位置参数和
    关键字参数。
62     - *args 允许函数接受任意数量的位置参数，并将它们收集到一个元组中。在函数调用时，传递
    的位置参数会被打包成一个元组，并赋值给 args 变量。
63     - **kwargs 允许函数接受任意数量的关键字参数，并将它们收集到一个字典中。在函数调用
    时，传递的关键字参数会被打包成一个字典，并赋值给 kwargs 变量。
64     在这里，arun 方法使用 *args 和 **kwargs 来允许传递任意数量的位置参数和关键字参数给
    被装饰的函数。这使得 arun 方法非常灵活，可以适应各种不同的实验函数，而不需要预先知道它们的
    参数结构。
65
66     magic_box(1, 2, "苹果", "香蕉", 100, color="红色", price=99.9) 后两个就叫关键字
    参数
67
68     """
69     async def arun(
70         self,
71         dataset: Dataset,
72         name: t.Optional[str] = None,
73         backend: t.Optional[t.Union[BaseBackend, str]] = None,
74         *args,
75         **kwargs,
76     ) -> "Experiment":
77         """Run the experiment against a dataset."""
78         # Generate name if not provided
79         if name is None:
80             name = memorable_names.generate_unique_name()
81         if self.name_prefix:
82             name = f"{self.name_prefix}-{name}"
83
84         # Resolve backend
85         experiment_backend = backend or self.default_backend
86         if experiment_backend:
87             resolved_backend = Experiment._resolve_backend(experiment_backend)
88         else:
89             resolved_backend = dataset.backend
90

```

```

88     # Create experiment
89     experiment_view = Experiment(
90         name=name,
91         data_model=self.experiment_model,
92         backend=resolved_backend,
93     )
94     """
95     调用一个async def的函数，只有前面加上await才会开始运行
96     """
97     # Create tasks for all items
98     tasks = []
99     for item in dataset:
100         tasks.append(self(item, *args, **kwargs))
101
102     progress_bar = None
103     try:
104         progress_bar = tqdm(total=len(dataset), desc="Running experiment")
105         """
106         开启并发执行所有任务，并在每个任务完成时更新进度条。
107         - asyncio.as_completed(tasks) 返回一个迭代器，产生已完成的任务的结果。
108         - 每当一个任务完成时，我们就可以获取它的结果并更新进度条，而不必等待所有任
109         务完成。
110         """
111         # Process all items
112         for future in asyncio.as_completed(tasks):
113             try:
114                 result = await future
115                 if result is not None:
116                     experiment_view.append(result)
117             except Exception as e:
118                 # Log individual task failures but continue
119                 print(f"Warning: Task failed with error: {e}")
120             finally:
121                 progress_bar.update(1)
122
123     finally:
124         if progress_bar:
125             progress_bar.close()
126
127     # Save experiment
128     experiment_view.save()
129
130     return experiment_view
131 """
132 t.Callable[[A], B]: 表示“一个可以调用的对象（函数）”，它接收 A 类型的参数，返回 B 类型
    的结果。

```

```

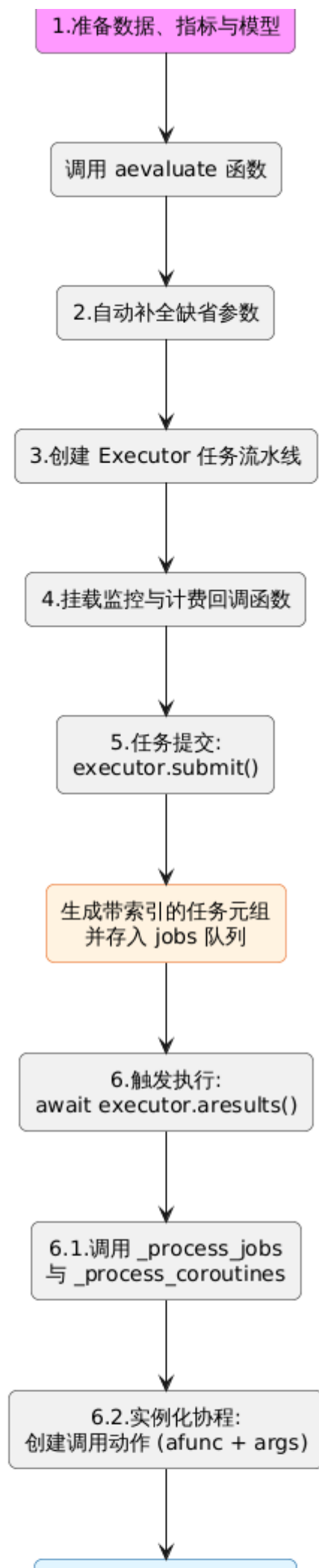
133 在这里，A 是 t.Callable（被装饰的函数）。
134 在这里，B 是 ExperimentProtocol（装饰后的结果）。
135 """
136
137 def experiment(
138     experiment_model: t.Optional[t.Type[BaseModel]] = None,
139     backend: t.Optional[t.Union[BaseBackend, str]] = None,
140     name_prefix: str = "",
141 ) -> t.Callable[[t.Callable], ExperimentProtocol]:
142     """Decorator for creating experiment functions.
143
144     Args:
145         experiment_model: The Pydantic model type to use for experiment results
146         backend: Optional backend to use for storing experiment results
147         name_prefix: Optional prefix for experiment names
148
149     Returns:
150         Decorator function that wraps experiment functions
151
152     Example:
153         @experiment(ExperimentDataRow)
154         async def run_experiment(row: TestDataRow):
155             # experiment logic here
156             return ExperimentDataRow(...)
157     """
158
159     def decorator(func: t.Callable) -> ExperimentProtocol:
160         wrapper = ExperimentWrapper(
161             func=func,
162             experiment_model=experiment_model,
163             default_backend=backend,
164             name_prefix=name_prefix,
165         )
166         return t.cast(ExperimentProtocol, wrapper)
167
168     return decorator

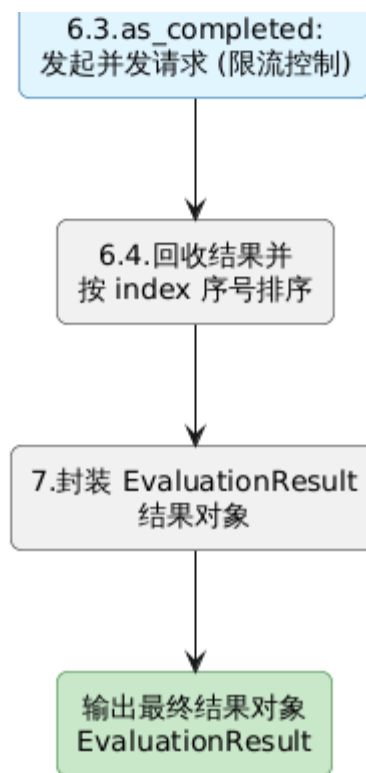
```

3. Agent 核心评估指标

3.1 Agent 评估的整体执行流水线

3.1.1 老版 evaluation





1. 准备评估数据（多轮对话）评委模型、Embedding 模型、评估指标并调用 aevaluate 函数
2. 自动补全没提供给 aevaluate 的参数
3. 创建任务流水线

Code block

```
1     executor = Executor(  
2         desc="Evaluating",  
3         keep_progress_bar=True,  
4         raise_exceptions=raise_exceptions,  
5         run_config=run_config,  
6         show_progress=show_progress,  
7         batch_size=batch_size,  
8         pbar=_pbar,  
9     )
```

4. 挂载和监控、计费相关的回调函数
5. 提交评估任务到执行队列

Code block

```
1     executor.submit(  
2         metric.multi_turn_ascore,  
3         sample,  
4         row_group_cm,  
5         name=f"{metric.name}-{i}",  
6         timeout=run_config.timeout,
```

```
7 )  
8 for metric in metrics
```

向 `executor.jobs` 中加入 `((metric.multi_turn_ascore, jobs_processed), (sample, row_group_cm), run_config.timeout, name)`

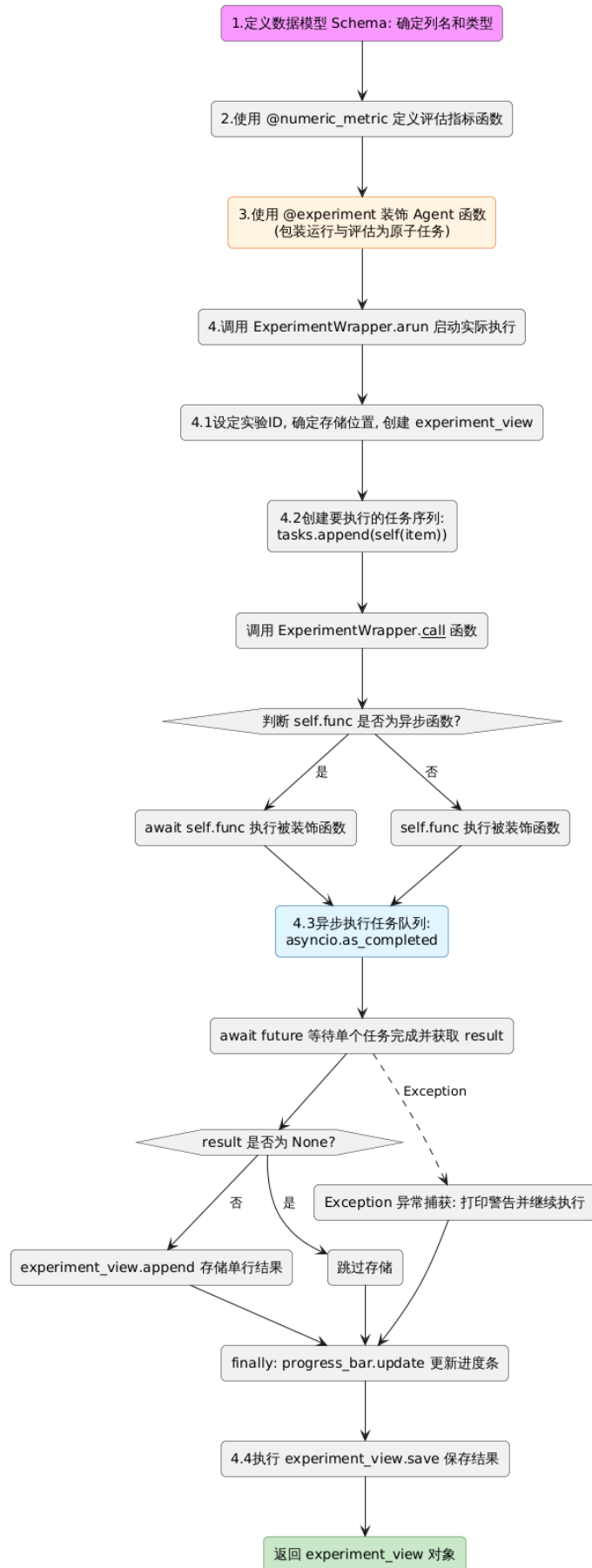
6. 通过 `await executor.aresults()` 这步触发已提交任务的实际执行

- a. 先调用 `_process_jobs(self)`，然后进行是否分批处理等判断后，调用 `_process_coroutines` 来发起网络请求、处理并发、并回收结果。
- b. 在 `_process_coroutines` 中
 - 通过 `coroutines = [afunc(*args, **kwargs) for afunc, args, kwargs, _ in jobs]` 创建调用包装过的 `metric.multi_turn_ascore` 的动作
 - 然后通过 `as_completed` 使得上述调用的动作实际执行
- c. 把结果按序号排序返回

7. 封装最终结果对象 `EvaluationResult`

3.1.2 新版 `experiment`

Experiment 运行与并发评估执行流程



1. 定义实验结果的列名和类型

就是 arun 中的 data_model

```

1 experiment_view = Experiment(
2     name=name,
3     data_model=self.experiment_model,
4     backend=resolved_backend,
5 )

```

2. 使用 `@numeric_metric` 把一个函数定义成一个评估指标，而不用像之前那样通过继承 `MultiTurnMetric` 的方式来定义指标了
3. 使用 `@experiment` 装饰评估 Agent 的函数，把运行 Agent 和评估结果包装成任务
4. 调用 `ExperimentWrapper` 中的 `arun` 函数实际执行任务。在 `arun` 中：
 - a. 设定该次实验的ID、确定评估结果存储的位置、创建存储结果的变量 `experiment_view`
 - b. 创建要执行的任务序列

Code block

```

1 tasks = []
2 for item in dataset:
3     tasks.append(self(item, *args, **kwargs))

```

这里的 `self` 实际上会调用 `ExperimentWrapper` 中的 `__call__` 函数

Code block

```

1 async def __call__(self, *args, **kwargs) -> t.Any:
2     """Call the original function."""
3     if asyncio.iscoroutinefunction(self.func):
4         return await self.func(*args, **kwargs)
5     else:
6         return self.func(*args, **kwargs)

```

而这里的 `func` 就是被 `@experiment` 装饰的函数

- c. 实际异步执行刚刚创建的任务队列：

Code block

```

1 progress_bar = tqdm(total=len(dataset), desc="Running experiment")
2
3 # Process all items
4 for future in asyncio.as_completed(tasks):
5     try:
6         result = await future
7         if result is not None:

```

```

8         experiment_view.append(result)
9     except Exception as e:
10         # Log individual task failures but continue
11         print(f"Warning: Task failed with error: {e}")
12     finally:
13         progress_bar.update(1)

```

d. 最后保存评估结果，返回 `experiment_view`

3.2 ToolCallAccuracy 详细设计

3.2.1 功能介绍

评估 Agent 在多轮对话中，执行工具调用的“动作”是否与预期的“标准答案”一致。

3.2.2 设计模式

策略模式：

- 实现：成员变量 `arg_comparison_metric` 可以实例化为不同的参数评分策略。
- 价值：将如何比对两个值的策略解耦。开发者可以随时注入 `SemanticSimilarity` 来替换硬核的 `ExactMatch`，而不需要修改 `ToolCallAccuracy` 的主体逻辑。

3.2.3 代码实现

设计了 `class ToolCallAccuracy(MultiTurnMetric)` 其中：

- 成员变量
 - a. `name`: "tool_call_accuracy" 指标名称
 - b. `strict_order`: 如果值为 `True`，则 Agent 调用工具的顺序必须和标准答案一模一样，如果为 `False` 则不关注顺序
 - i. `_required_columns`: 数据列依赖声明
 - d. `arg_comparison_metric`: 参数比对的子指标
- 核心函数: `_multi_turn_ascore`
 - a. 首先先得到 AI 发出的工具调用和答案说明的工具调用: `pred_tool_calls` 和 `reference_tool_calls`
 - b. 先预处理:
 - 答案里没调用工具，ai也没调用工具，则得1分
 - 答案里会调用工具，ai没调用工具，则得0分
 - 答案里没调用工具，但ai调用工具，则得0分

- c. 根据 strict_order 来判定是否要对 pred_tool_calls 和 reference_tool_calls 进行排序
- d. 先比对调用的工具，对于严格模式，如果顺序或名称不对就直接得 0 分

Code block

```
1  def is_sequence_aligned(  
2      self, pred_sequence: t.List[str], ref_sequence: t.List[str]  
3  ) -> bool:  
4      if self.strict_order:  
5          return pred_sequence == ref_sequence  
6      else:  
7          # For non-strict mode, sort both sequences before comparison  
8          return sorted(pred_sequence) == sorted(ref_sequence)  
9  
10 // _multi_turn_ascore  
11 sequence_aligned = int(  
12     self.is_sequence_aligned(tool_call_pred_sequence,  
13     tool_call_ref_sequence)  
14 )  
15 // ...  
16 return score * sequence_aligned
```

如对于输入：

Code block

```
1  data = {  
2      "user_input": [  
3          [  
4              HumanMessage(content="我想订一张去上海的机票。").model_dump(),  
5              AIMessage(  
6                  content="没问题。",  
7                  tool_calls=[  
8                      ToolCall(name="weather", args={"destination":  
9                          "Shanghai"}),  
10                     ToolCall(name="book_flight", args={"destination":  
11                         "Shanghai"}),  
12                 ]  
13             ).model_dump()  
14         ],  
15         "reference_tool_calls": [  
16             [  
17                 ToolCall(name="weather1", args={"destination":  
18                     "Shanghai"}).model_dump(),
```

```

17         ToolCall(name="book_flight", args={"destination":
    "Shanghai"}).model_dump(),
18     ]
19 ]
20 }

```

实际会得到：{'tool_call_accuracy': 0.0000}

e. 再分别对比每一个参数

Code block

```

1  for ref_tool_call, pred_tool_call in zip(reference_tool_calls,
    pred_tool_calls):
2      if ref_tool_call.name == pred_tool_call.name:
3          arg_score = await self._get_arg_score(
4              pred_tool_call.args, ref_tool_call.args, callbacks
5          )
6          score += arg_score
7
8  score /= len(reference_tool_calls)

```

总的来看就是在调用的工具名正确的情况下获得每一组参数的得分并累加，最后再除以 len(reference_tool_calls)。对于具体一组的参数的评分：

Code block

```

1  async def _get_arg_score(
2      self, preds: t.Dict[str, t.Any], refs: t.Dict[str, t.Any],
3      callbacks: Callbacks
4  ) -> float:
5      if not refs and not preds:
6          return 1.0
7      if not refs:
8          return 0.0
9
10     score = 0.0
11     for arg in refs.keys():
12         if arg in preds:
13             score += await self.arg_comparison_metric.single_turn_ascore(
14                 SingleTurnSample(
15                     response=str(preds[arg]), reference=str(refs[arg])
16                 ),
17                 callbacks,
18             )

```

```
19         return score / len(refs.keys())
```

i. 预处理:

- 都不需要参数, Agent也没传参数, 则得1分
- 标准答案没参数, Agent却瞎传了参数, 则得0分

ii. 根据设定好的 `arg_comparison_metric` 来对参数比对进行打分, 默认是:

Code block

```
1 arg_comparison_metric: SingleTurnMetric = field(  
2     default_factory=lambda: ExactMatch()  
3 )
```

就是严格比对字符串

iii. 最后计算一个参数组中的平均分

f. 再次对分数进行惩罚

Code block

```
1 if compared_count < len(reference_tool_calls):  
2     coverage_penalty = compared_count / len(reference_tool_calls)  
3     score *= coverage_penalty
```

对 score 乘上 $\min(\text{len}(\text{pred_tool_calls}), \text{len}(\text{reference_tool_calls})) / \text{len}(\text{reference_tool_calls})$ 加权

但是这里个人觉得多惩罚了一次, 因为之前就已经通过 $\text{score} / \text{len}(\text{reference_tool_calls})$ 加权了

3.2.4 问题

对于参数判定时的:

Code block

```
1 compared_count = min(len(pred_tool_calls), len(reference_tool_calls))
```

和 后续的再次惩罚:

Code block

```
1 if compared_count < len(reference_tool_calls):
```

```
2     coverage_penalty = compared_count / len(reference_tool_calls)
3     score *= coverage_penalty
```

这两样是否是冗余的？

具体说明：首先可以知道如果 `sequence_aligned == 0` 结果肯定就为0，无论后续的参数评分或加权或惩罚是多少最终的分数肯定为0，所以在这种情况下问题中提到的两部分逻辑是冗余的。而当 `sequence_aligned == 1` 时，无论是否是严格模式，都可以保证至少 `len(pred_tool_calls) == len(reference_tool_calls)`，那在这个情况下，`compared_count` 就是冗余的，`compared_count == reference_tool_calls` 一定成立。那这就会导致 `if compared_count < len(reference_tool_calls)` 这个分支永远不会执行，逻辑冗余。

3.3 ToolCallF1 详细设计

3.3.1 功能介绍

把 Agent 调用的所有工具看作一个集合，通过 F1-Score 来评估工具调用的全面性和准确性。

3.3.2 设计模式

无

3.3.3 代码实现

设计了 `class ToolCallF1(MultiTurnMetric)` 其中：

- 成员变量
 - a. `name`: "tool_call_f1" 指标名称
 - b. `batch_size`: 单次批处理的样本数量
 - c. `is_multi_turn`: 默认 `True`，支持多轮对话
 - d. `_required_columns`: 表示输入中必须包含的字段

Code block

```
1 // 默认值
2 {
3     MetricType.MULTI_TURN: {
4         "reference_tool_calls",
5         "user_input",
6     }
7 }
```

- 核心函数: `_multi_turn_ascore`

- a. 创建答案和实际的工具调用集合，集合元素为： `(call.name, _make_hashable(call.args))`

Code block

```
1 expected: set[tuple[str, frozenset]] = set()
2 if sample.reference_tool_calls:
3     for call in sample.reference_tool_calls:
4         print("[ToolCallF1] [ref]:", call)
5         expected.add((call.name, _make_hashable(call.args)))
6
7 actual: set[tuple[str, frozenset]] = set()
8 for msg in sample.user_input:
9     if isinstance(msg, AIMessage) and msg.tool_calls is not None:
10        for call in msg.tool_calls:
11            print("[ToolCallF1] [act]:", call)
12            actual.add((call.name, _make_hashable(call.args)))
```

- b. 求 tp、fp、fn

Code block

```
1 tp = len(actual & expected)
2 fp = len(actual - expected)
3 fn = len(expected - actual)
```

- c. 求 precision、recall

Code block

```
1 precision = tp / (tp + fp) if (tp + fp) > 0 else 0.0
2 recall = tp / (tp + fn) if (tp + fn) > 0 else 0.0
```

- d. 求 f1

Code block

```
1 f1 = (
2     2 * precision * recall / (precision + recall)
3     if (precision + recall) > 0
4     else 0.0
5 )
```


3.4 AgentGoalAccuracy 详细设计

在Agent评测中，仅仅对工具使用的准确性进行检测是远远不够的，核心在于Agent是否真正解决了用户的问题。在Ragas框架中，Agent目标标准度是一个二元指标，得分为1或0，用于评估LLM智能体在多轮对话中识别并达成用户目标的表现。

Ragas将该指标放在业务实现包ragas.metrics.collections中，拆分为两个核心子类，分别应对不同数据闭环下的评估场景。

3.4.1 AgentGoalAccuracyWithReference 有参考结果的评估

在具备测试基准的场景下，AgentGoalAccuracyWithReference类通过对比Agent轨迹的“最终状态”与给定的参考结果来进行评估，底层逻辑是将真实参考值与Agent的MultiTurnSample送入LLM进行严格比对。

Code block

```
1  class AgentGoalAccuracyWithReference(BaseMetric):
2
3      llm: "InstructorBaseRagasLLM"
4
5      def __init__(
6          self,
7          llm: "InstructorBaseRagasLLM",
8          name: str = "agent_goal_accuracy",
9          **kwargs,
10     ):
11         self.llm = llm
12         self.workflow_prompt = InferGoalOutcomePrompt()
13         self.compare_outcome_prompt = CompareOutcomePrompt()
14
15         super().__init__(name=name, **kwargs)
16
17     async def ascore(
18         self,
19         user_input: List[Union[HumanMessage, AIMessage, ToolMessage]],
20         reference: str,
21     ) -> MetricResult:
22
23         if not isinstance(user_input, list):
24             raise ValueError("user_input must be a list of messages")
25         if not reference:
26             raise ValueError(
27                 "reference must be provided for AgentGoalAccuracyWithReference"
28             )
29
```

```

30         conversation = self._format_conversation(user_input)
31
32         # Step 1: Infer the end state from the workflow
33         workflow_result = await self._infer_goal_outcome(conversation)
34
35         # Step 2: Compare the end state with reference
36         verdict = await self._compare_outcomes(reference,
37         workflow_result.end_state)
38
39         return MetricResult(value=float(verdict))
40
41     def _format_conversation(
42         self, messages: List[Union[HumanMessage, AIMessage, ToolMessage]]
43     ) -> str:
44         """Format messages into a readable conversation string."""
45         lines = []
46         for msg in messages:
47             lines.append(msg.pretty_repr())
48         return "\n".join(lines)
49
50     async def _infer_goal_outcome(self, conversation: str) -> WorkflowOutput:
51         """Infer the user goal and end state from the conversation."""
52         input_data = WorkflowInput(workflow=conversation)
53         prompt_str = self.workflow_prompt.to_string(input_data)
54         return await self.llm.agenerate(prompt_str, WorkflowOutput)
55
56     async def _compare_outcomes(self, desired: str, arrived: str) -> int:
57         """Compare desired outcome with achieved outcome."""
58         input_data = CompareOutcomeInput(
59             desired_outcome=desired, arrived_outcome=arrived
60         )
61         prompt_str = self.compare_outcome_prompt.to_string(input_data)
62         result = await self.llm.agenerate(prompt_str, CompareOutcomeOutput)
63         return int(result.verdict)

```

3.4.2 AgentGoalAccuracyWithoutReference 无参考结果的评估

然而在真实的生产环境中往往没有标准答案，于是该类利用LLM的逻辑推理能力进行盲评，其底层执行流水线分为三步：

1. 意图推断：通过InferGoalOutcomePromt提取用户的潜在目标。
2. 结果提取：解析Agent的多轮交互轨迹，总结实际完成的工作。
3. 结果比对：通过CompareOutcomePrompt对比意图与结果，完美覆盖则判为1，否则为0。

```

1  class AgentGoalAccuracyWithoutReference(BaseMetric):
2
3      llm: "InstructorBaseRagasLLM"
4
5      def __init__(
6          self,
7          llm: "InstructorBaseRagasLLM",
8          name: str = "agent_goal_accuracy",
9          **kwargs,
10     ):
11         self.llm = llm
12         self.workflow_prompt = InferGoalOutcomePrompt()
13         self.compare_outcome_prompt = CompareOutcomePrompt()
14
15         super().__init__(name=name, **kwargs)
16
17     async def ascore(
18         self,
19         user_input: List[Union[HumanMessage, AIMessage, ToolMessage]],
20     ) -> MetricResult:
21         if not isinstance(user_input, list):
22             raise ValueError("user_input must be a list of messages")
23
24         conversation = self._format_conversation(user_input)
25
26         # Step 1: Infer the user goal and end state from the workflow
27         workflow_result = await self._infer_goal_outcome(conversation)
28
29         # Step 2: Compare the inferred goal with the end state
30         verdict = await self._compare_outcomes(
31             workflow_result.user_goal, workflow_result.end_state
32         )
33
34         return MetricResult(value=float(verdict))
35
36     def _format_conversation(
37         self, messages: List[Union[HumanMessage, AIMessage, ToolMessage]]
38     ) -> str:
39         """Format messages into a readable conversation string."""
40         lines = []
41         for msg in messages:
42             lines.append(msg.pretty_repr())
43         return "\n".join(lines)
44
45     async def _infer_goal_outcome(self, conversation: str) -> WorkflowOutput:
46         """Infer the user goal and end state from the conversation."""
47         input_data = WorkflowInput(workflow=conversation)

```

```

48         prompt_str = self.workflow_prompt.to_string(input_data)
49         return await self.llm.agenerate(prompt_str, WorkflowOutput)
50
51     async def _compare_outcomes(self, desired: str, arrived: str) -> int:
52         """Compare desired outcome with achieved outcome."""
53         input_data = CompareOutcomeInput(
54             desired_outcome=desired, arrived_outcome=arrived
55         )
56         prompt_str = self.compare_outcome_prompt.to_string(input_data)
57         result = await self.llm.agenerate(prompt_str, CompareOutcomeOutput)
58         return int(result.verdict)

```

3.5 其他核心 Agent 评估指标的分析

除了目标是否达成，生产级Agent所面临的另一大挑战是安全边界与领域控制。如果用户提出了不在Agent的预定义的主题领域内的问题，Agent就不应该进行回答，否则会带来极大的业务风险。在Ragas的collections模块中，这由主题一致性(Topic Adherence)来衡量。

1. 主题一致性的设计与底层解析

该指标用于评估Agent在多轮对话中是否严格保持在预定义的主题领域内，并在面对越界请求时是否进行了适当的拒绝，在源码的注释中作者进行了清楚的说明：

Code block

```

1  class TopicAdherence(BaseMetric):
2      """
3      Measures how well an AI system adheres to predefined topics during
4      conversations.
5
6      AI systems deployed in real-world applications are expected to stay within
7      domains
8      of interest. This metric evaluates the ability of the AI to only answer
9      queries
10     related to predefined topics and refuse queries outside those topics.
11     """

```

TopicAdherence的处理逻辑则在ascore()中。ascore接受用户输入和允许的主题作为参数，接着从对话中提取主题并检查哪些问题得到AI的回答，然后将得到回答的主题和允许回答主题进行比对，最终经过计算得到主题一致性维度的得分。

Code block

```

1  async def ascore(
2      self,
3      user_input: List[Union[HumanMessage, AIMessage, ToolMessage]],

```

```

4         reference_topics: List[str],
5     ) -> MetricResult:
6         """
7         Calculate topic adherence score.
8
9         Args:
10             user_input: List of conversation messages
11             reference_topics: List of allowed topics the AI should adhere to
12
13         Returns:
14             MetricResult with topic adherence score (0.0-1.0, higher is better)
15         """
16         if not isinstance(user_input, list):
17             raise ValueError("user_input must be a list of messages")
18         if not isinstance(reference_topics, list) or not reference_topics:
19             raise ValueError("reference_topics must be a non-empty list of
20 topics")
21
22         # Format conversation as pretty string
23         conversation = self._format_conversation(user_input)
24
25         # Step 1: Extract topics from the conversation
26         topics = await self._extract_topics(conversation)
27         if not topics:
28             return MetricResult(value=float("nan"))
29
30         # Step 2: Check which topics the AI answered vs refused
31         topic_answered = await self._check_topics_answered(conversation,
32 topics)
33
34         # Step 3: Classify topics against reference topics
35         topic_classifications = await self._classify_topics(reference_topics,
36 topics)
37
38         # Step 4: Compute score based on mode
39         score = self._compute_score(topic_answered, topic_classifications)
40
41         return MetricResult(value=float(score))

```

4. 实践收获和体会

4.1 深入理解大模型 Agent 的评测理论与工程化实现

通过阅读 Ragas 源码发现了它从离线对比向运行后直接评测动态 workflows 演进的设计。在工程实现上，框架利用装饰器模式实现了业务逻辑与评测任务的封装，比之前使用 evaluate 堆成一坨更解耦。

在评估指标的设计部分，如 ToolCallAccuracy，策略模式将不同的打分准则与执行流程解耦，支持字符串比对、语义理解的切换。针对大模型 API 的高延迟特性，源码采用异步并发配合信号量限流又增加了评估效率又防止一次请求太多 API 帐号被封。

4.2 提升 Python 高质量开源项目的阅读与分析能力

通过这次对 Ragas 源码的阅读与分析，我在 Python 工程化开发和复杂系统架构认知上获得了长足的进步，具体表现在以下三点。

1. 目录解耦与开闭原则

一开始阅读源码时，我在 ragas.metrics 中没有找到具体的 Agent 评估类，后来才发现 Ragas 将抽象层与业务实现层 (ragas.metrics.collections) 进行了彻底的分离。这种设计允许未来的开发者自由地向 collections 中添加新指标，而不会破坏核心调度层的稳定性，体现了软件系统设计中“对扩展开放，对修改封闭”的开闭原则。

2. 数据类规范与标准化流转

Ragas 摒弃了早期使用松散的 Dictionary 传递数据的做法，全面引入了 MultiTurnSample、HumanMessage、AIMessage 等强类型约束的数据对象。这不仅极大提升了代码的可读性和 IDE 的提示友好度，也使得框架能无缝对接底层的 pandas 和 HuggingFace Dataset，为我以后编写 Python 数据处理树立了标杆。

3. 异步编程在大模型评测中的应用

在阅读中我注意到所有 Ragas 指标的核心计算入口统一变为了 ascore() 异步打分。由于 LLM 评测本质上是高密度的 I/O 密集型网络请求任务，Ragas 底层巧妙使用了 async/await 和 Executor 的并发与重试控制，彻底解决了海量测评时导致的主线程阻塞问题。这补足了我以往在处理大模型 API 并发时的技术盲区。

4.3 对开源协作与代码规范的认识

通过本次对 Ragas 源码的深度泛读与分析，我们在开源社区协作机制和工业级 Python 代码规范上都有了极其深刻的认识，也对我们小组成员的日常编码产生了很大的裨益：

1. 强类型约束与面向对象封装

在以往的 Python 脚本开发中，一个习惯是依赖 Dictionary 进行松散的数据传参。而在 Ragas 框架中，我看到了强类型注解 (Type Hints) 的全面应用。框架通过 `ragas.dataset_schema` 定义了严格的数据实体类 (如 `MultiTurnSample`、`ToolCall` 等)。这种强类型约束不仅使复杂 Agent 轨迹的数据流转一目了然，极大提升了代码在 IDE 中的智能提示与可读性，更从根本上规避了运行时因键值拼写错误而导致的隐蔽 Bug。

2. 灵活运用设计模式践行“开闭原则”

Ragas 卓越的软件架构生动展示了什么是“对扩展开放，对修改封闭”——例如在实现 `ToolCallAccuracy` 指标时，作者采用了**策略模式**，允许开发者随时将底层变量 `arg_comparison_metric` 实例化为不同的参数比对策略 (从 `ExactMatch` 字符串匹配无缝切

换到基于大模型的 `SemanticSimilarity` 语义匹配)。这种解耦设计意味着，当未来需要引入新的对比算法时，只需实现一个新的策略子类即可，核心评估流水线的代码无需任何改动。

3. 开源协作的透明度与生态共建

在分析 Ragas 底层 Executor 的异步执行以及网络防阻塞重试机制时，我们深刻意识到，这些健壮的工程化解决方案往往是开源社区在处理海量大模型并发请求时，历经反复试错后沉淀出的最佳实践。开源平台（如 GitHub Issues 和 Pull Requests）为这些技术难点提供了透明开放的交流池。同时，Ragas 将底层抽象接口与具体的业务指标实现层（`collections` 包）彻底剥离，极大地降低了外部开发者贡献新指标的门槛。这说明优秀的开源项目不仅是分享一堆代码，更是通过出色的架构设计去引导和鼓励一个活跃的社区生态进行高效共建。

5. 大模型辅助代码阅读的实践认识

5.1 使用的大模型辅助工具

在本次软件开发实践活动中，本小组借助了多种大模型工具来辅助开展工作，具体包括 GitHub copilot, deepseek, claude code, Google AI Studio等。这些工具各具特色，为我们提供了丰富的功能支持，助力源码阅读实践的顺利推进。

5.2 大模型辅助开源阅读的成效

本小组主要利用大模型工具在以下几个方面的源码阅读实践工作中发挥了重要作用：

1. 代码理解：

本项目基于python编写，有丰富的特性使用，尤其在类型检查方面下了功夫。由于我们平时对于pydantic等与类型检查相关的包了解不深，在面对复杂的代码逻辑时，大模型工具能够快速帮助我们理解代码的功能和结构。例如，当遇到一段晦涩难懂的实现代码时，通过向大模型工具提问，它能够以通俗易懂的语言解释代码的运行流程和关键点，让我们迅速把握代码的核心。同时还可以给出简洁的代码特性使用示例，便于我们自主运行代码学习语言特性。

- 代码中关于泛型的使用。由于python本身是个动态语言，所谓的泛型在运行中也会被擦除，相当于泛型是面向编码人员或者说阅读代码的人员。这与其他静态语言有着显著不同，因此我们在阅读相关代码会产生疑惑，大模型可以在此时给出详细解释。并且可以带我们实践。

那为什么一个有泛型一个没有呢？

[experiment.py](#) [dataset.py](#)

你这个问题很关键，我来直接解释“为什么 `Dataset` 有泛型而 `Experiment` 没有”。

- ☒ 先给结论
- ☒ 再对应到代码位置说明
- ☒ 最后给你一个类型设计上的直觉

结论：这是类型设计取舍，不是能力差异。`Experiment` 运行时同样支持模型校验，但在静态类型层面被故意写得更宽松。

- 在 `src/ragas/dataset.py` 里，`DataTable` 是 `Generic[T]`，`Dataset` 定义为 `class Dataset(DataTable[T])`。这让 `Dataset` 的行类型在类型检查时更稳定（更像“已知 schema 的输入表”）。
- 在 `src/ragas/experiment.py` 里，`Experiment` 是 `class Experiment(DataTable)`（没带 `T`），因为实验结果来源于被装饰函数，返回类型在框架层面被允许更灵活（`dict/BaseModel`，且 `experiment_model` 可选）。
- `ExperimentWrapper.__call__` 返回 `t.Any`，`arun` 内部也是把任务结果逐条 `append`；这条链路本身就偏动态。如果把 `Experiment` 强行泛型化，现有 `Protocol` 和 decorator 签名会更复杂，用户侧收益未必大。
- 换句话说：`Dataset` 更偏“输入契约固定”，`Experiment` 更偏“执行产物可配置/可变”，所以一个强类型提示更强，一个故意放宽。
- 运行时两者都走 `data_model` 做校验（`DataTable.append / save` 逻辑），所以“没泛型”不代表“不能校验”，主要影响的是 IDE/pyright 的静态提示精度。

如果你愿意，我可以下一步给你一个“把 `Experiment` 也改成泛型”的最小改造草案（含 `Protocol` 签名怎么改），你就能看到为什么现在作者可能选择不这么做。

☒ Completed

GPT-5.3-Codex · 0.9x  

第三部分：带你用 Python 实践泛型

打开你的编辑器（推荐 VSCode + Pylance 插件，或者 PyCharm），我们来写一段完整的代码，体会泛型的威力。

Step 1: 准备基础类

我们先模拟一个场景：你有不同的数据模型（用户、商品），它们都继承自 `BaseModel`。

<> Python

```
class BaseModel:
    def __init__(self, id: int):
        self.id = id

class User(BaseModel):
    def __init__(self, id: int, username: str):
        super().__init__(id)
        self.username = username

class Product(BaseModel):
    def __init__(self, id: int, price: float):
        super().__init__(id)
        self.price = price
```

Step 2: 定义泛型 `DataTable`

我们要写一个 `DataTable` 类，它可以用来存 `User`，也可以用来存 `Product`。

<> Python

- 代码中修饰器的使用。修饰器是一个有趣的语法糖，但在实际代码中我们看到了修饰器工厂这种有别于普通修饰器的代码。但第一次遇见后总会让人摸不清头脑，这时就可以获取大模型的帮助。

Code block

1 `@experiment()`


```
2  async def run_experiment(row):
3      response = rag_client.query(row["question"])
4      ...
5      return experiment_view
```

解释一下这个装饰器，我感觉和之前见到的不太一样

Model • 7:22 PM

✦ Thoughts

Expand to view model thoughts

你的直觉非常敏锐！这个装饰器确实和大部分 Python 教程里教的“标准装饰器”**截然不同**。

标准的教科书装饰器通常长这样（两层嵌套，内部用 `*args`, `**kwargs` 包装）：

<> Python

📄 📋 ^

```
def simple_decorator(func):
    def wrapper(*args, **kwargs):
        print("执行前")
        result = func(*args, **kwargs)
        print("执行后")
        return result
    return wrapper
```

但你提供的这个 `experiment` 装饰器，使用了两个非常高级的设计模式。为了让你彻底看透它，我们把它的“异常之处”拆解成三个核心点：

Start typing a prompt, use alt + enter to append



Tools



Run Ctrl ⌘

2. 文档生成：我们在阅读源码中发现metric中许多评价指标已经变成了DEPRECATED，为了迅速确定某个指标现在的状态，我们使用大模型工具检索目录，提取关键字，映射，可以快速获得一个metric对应文档，大幅度提升了我们的阅读效率。

Open terminal

✓ Read  _answer_relevance.py successfully.

✓ Read  metric.py successfully.

✓ Read  metric.py successfully.

✓ Edited ragas_metrics_api_mapping.txt with apply_patch tool

✓ Read  ragas_metrics_api_mapping.txt successfully.

收到，你要的是基于 **collections** 实际导出做核对，不要“想当然”映射。

我的处理清单：

- ✓ 逐项对比 **src/ragas/metrics/__init__.py** 里的 **_DEPRECATED_METRICS** (54)
- ✓ 严格对比 **src/ragas/metrics/collections/__init__.py** 的 **__all__** (39)
- ✓ 把“同名可迁移 / 无同名导出 / 改名候选”分开写清
- ✓ 更新清单文件到 **_experiments/ragas_metrics_api_mapping.txt**

已完成，并且这次是按源码实查，不是臆测。

关键结论：

- 同名可直接迁移：26
- 在 **collections.__all__** 找不到同名：28
- 文件已更新：**_experiments/ragas_metrics_api_mapping.txt**

```
40 =====
41 二、未在 collections.__all__ 找到同名导出
42 =====
43 answer_correctness
44 answer_relevancy
45 AnswerSimilarity
46 answer_similarity
47 AspectCritic
48 ChrfScore
49 context_entity_recall
50 context_precision
51 context_recall
52 faithfulness
53 FaithfulnesswithHHEM
54 IDBasedContextPrecision
55 IDBasedContextRecall
56 InstanceRubrics
57 LLMContextPrecisionWithoutReference
58 LLMContextPrecisionWithReference
59 LLMContextRecall
60 LLMSQLEquivalence
61 multimodal_faithness
62 multimodal_relevance
63 NonLLMContextPrecisionWithReference
64 NonLLMContextRecall
65 ResponseRelevancy
66 RubricsScore
67 SimpleCriteriaScore
68 SummarizationScore
69 summarization_score
70 TopicAdherenceScore
71 |
72 =====
73 三、仅基于 collections.__all__ 的改名候选（需你再确认语义）
74 =====
75 ChrfScore -> CHRFScore
76 LLMSQLEquivalence -> SQLSemanticEquivalence
77 SummarizationScore -> SummaryScore
78 TopicAdherenceScore -> TopicAdherence
79 InstanceRubrics -> InstanceSpecificRubrics
80 RubricsScore -> RubricsScoreWithReference / RubricsScoreWithoutReference
81
82 =====
83 四、统计
84 =====
85 _DEPRECATED_METRICS 总数：54
86 同名可迁移：26
87 无同名导出：28
88
89
```

5.3 遇到的问题与反思

用大模型工具辅助开展源码阅读实践工作，效果显著，但也存在一些问题：

- 收获：

- 阅读效率大幅提升：大模型工具在代码理解、文档生成以及技术问题解答等方面为我们节省了大量的时间和精力。让我们能更快的适应一个不熟悉语言编写的大型项目。
- 知识获取更加便捷：借助大模型工具，我们可以快速学习评估不同类型系统的关键点，并且可以轻松写出示例代码并运行，拓宽了我们的知识面，提升了本小组的技术水平。
- 问题：
 - 对上下文理解不够深入：类似Google AI Studio，我们在问问题时，很难提供完全的符合大模型需要的上下文，没能传达出其中复杂的关联关系，大模型工具可能无法准确把握某个代码片段在整个项目架构中的作用和依赖关系，导致给出的建议不够贴合实际项目需求。像Claude Code这类agent在这方面表现更好。
 - 依赖性：在使用大模型工具的过程中，我们可能会逐渐形成对工具的过度依赖。一旦遇到工具无法解决的问题，或者工具出现故障无法使用时，我们很可能会导致进度停滞。因此，我们需要在使用工具的同时，不断提升自身的专业技能和独立解决问题的能力。